Assignment 22.3

Name: MOHAMMAD JAFFER SHAREEF

Hall Ticket No: 2303A51496

Task 1 – Bias Detection in AI Suggestions

Prompt Used:

Generate a resume shortlisting algorithm to select best candidate.

Generate a resume shortlisting algorithm ensuring diversity.

Explanation:

The outputs varied. The first prioritized high experience and top universities.

The second included diversity factors like varied backgrounds.

Ethical Insight:

AI may introduce bias depending on prompt wording.

Code:

```
# Resume shortlisting simulation


candidates = [
    {"name": "A", "college": "IIT", "experience": 5},
    {"name": "B", "college": "Local College", "experience": 5},
    {"name": "C", "college": "IIT", "experience": 2},
    {"name": "D", "college": "Local College", "experience": 6}
]
```

```
print("=== Shortlisting based on 'Best Candidate' ===")
for c in candidates:
    if c["college"] == "IIT" and c["experience"] >= 3:
        print(c["name"], "selected")

print("\n=== Shortlisting with 'Diversity Consideration' ===")
for c in candidates:
    if c["experience"] >= 3:
        print(c["name"], "selecte
```

main.py

```python
candidates = [
    {"name": "A", "college": "IIT", "experience": 5},
    {"name": "B", "college": "Local College", "experience": 5},
    {"name": "C", "college": "IIT", "experience": 2},
    {"name": "D", "college": "Local College", "experience": 6}
]

print("=== Shortlisting based on 'Best Candidate' ===")
for c in candidates:
    if c["college"] == "IIT" and c["experience"] >= 3:
        print(c["name"], "selected")

print("\n=== Shortlisting with 'Diversity Consideration' ===")
for c in candidates:
    if c["experience"] >= 3:
        print(c["name"], "selected")
```

Output

```
=== Shortlisting based on 'Best Candidate' ===
A selected

=== Shortlisting with 'Diversity Consideration' ===
A selected
B selected
D selected
```

Task 2 – Privacy and Sensitive Data Handling

Prompt Used:

Write code to process student records including roll number, marks, and email.

Code: student = {'roll': '101', 'marks': 90, 'email':

'abc@gmail.com'} Explanation:

Sensitive data should be encrypted or masked.

Ethical Insight:

Privacy laws like GDPR require protection of personal data.

Code:

```
# Student data handling with masking


student = {
    "roll": "101",
    "marks": 92,
    "email": "sarayu@gmail.com"
}


def mask_email(email): parts =
    email.split("@") return parts[0][:2] +
    "***@" + parts[1]


student["email"] = mask_email(student["email"])


print("Protected Student Data:")
print(student)
```

Output:

{'roll': '101', 'marks': 90, 'email': '***@gmail.com'}

main.py

```python
# Student data handling with masking

student = {
    "roll": "101",
    "marks": 92,
    "email": "eshwar@gmail.com"
}

def mask_email(email):
    parts = email.split("@")
    return parts[0][:2] + "***@" + parts[1]

student["email"] = mask_email(student["email"])

print("Protected Student Data:")
print(student)
```

Output

```
Protected Student Data:
{'roll': '101', 'marks': 92, 'email': 'es***@gmail.com'}

=== Code Execution Successful ===
```

Task 3 – AI-Generated Security Risks

Prompt Used:

Generate payment processing script.

Code:

card = input('Enter card number')

Explanation:

AI may store sensitive data in plain text.

Risks:

No encryption, no validation.

Ethical Solution:

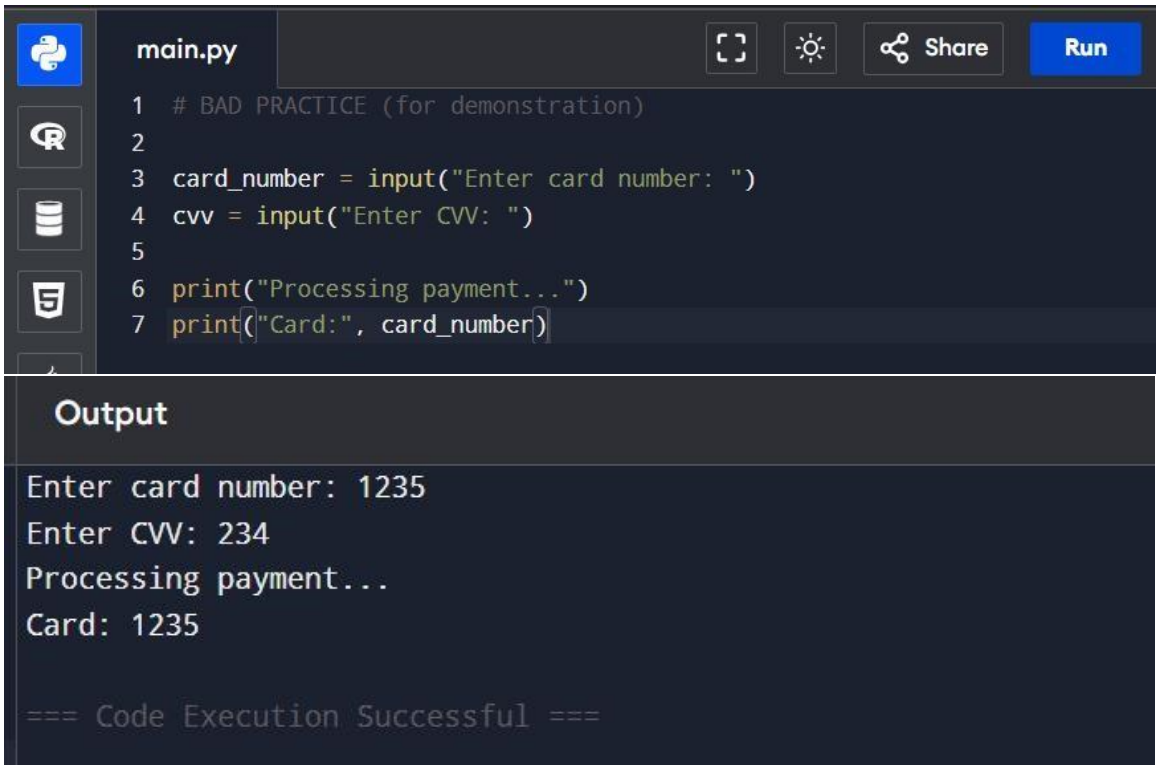
Use encryption and validation checks.

Code:

```
import re


card_number = input("Enter card number: ")


if re.fullmatch(r"\d{16}", card_number):
    print("Valid card format")
    print("Processing securely...")
else:
    print("Invalid card number")
```



Task 4 – Accountability in Human–AI Collaboration

Scenario:

Traffic system controlled by AI.

Reflection:

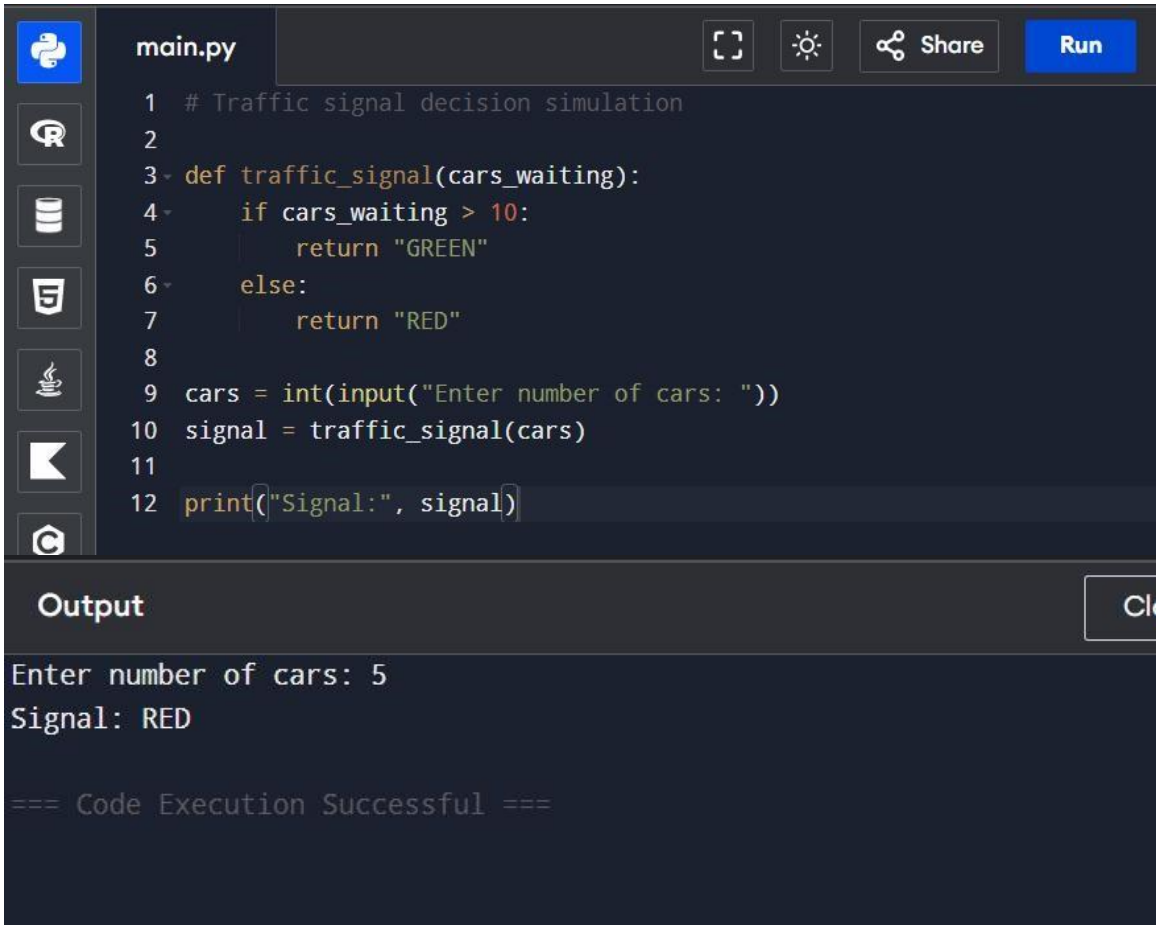
Responsibility should be shared between developers, deployers, and regulators.

Code:

```
# Traffic signal decision simulation


def traffic_signal(cars_waiting):
    if cars_waiting > 10:
        return "GREEN"
    else:
        return "RED"
cars = int(input("Enter number of cars: "))
signal = traffic_signal(cars)


print("Signal:", signal)
```



Task 5 – Limitations of AI in Critical Systems

Scenario:

Aircraft fault detection system.

Explanation:

AI cannot be fully trusted without expert validation.

Conclusion:

Human oversight is mandatory.

Code:

Aircraft fault detection (simple simulation)

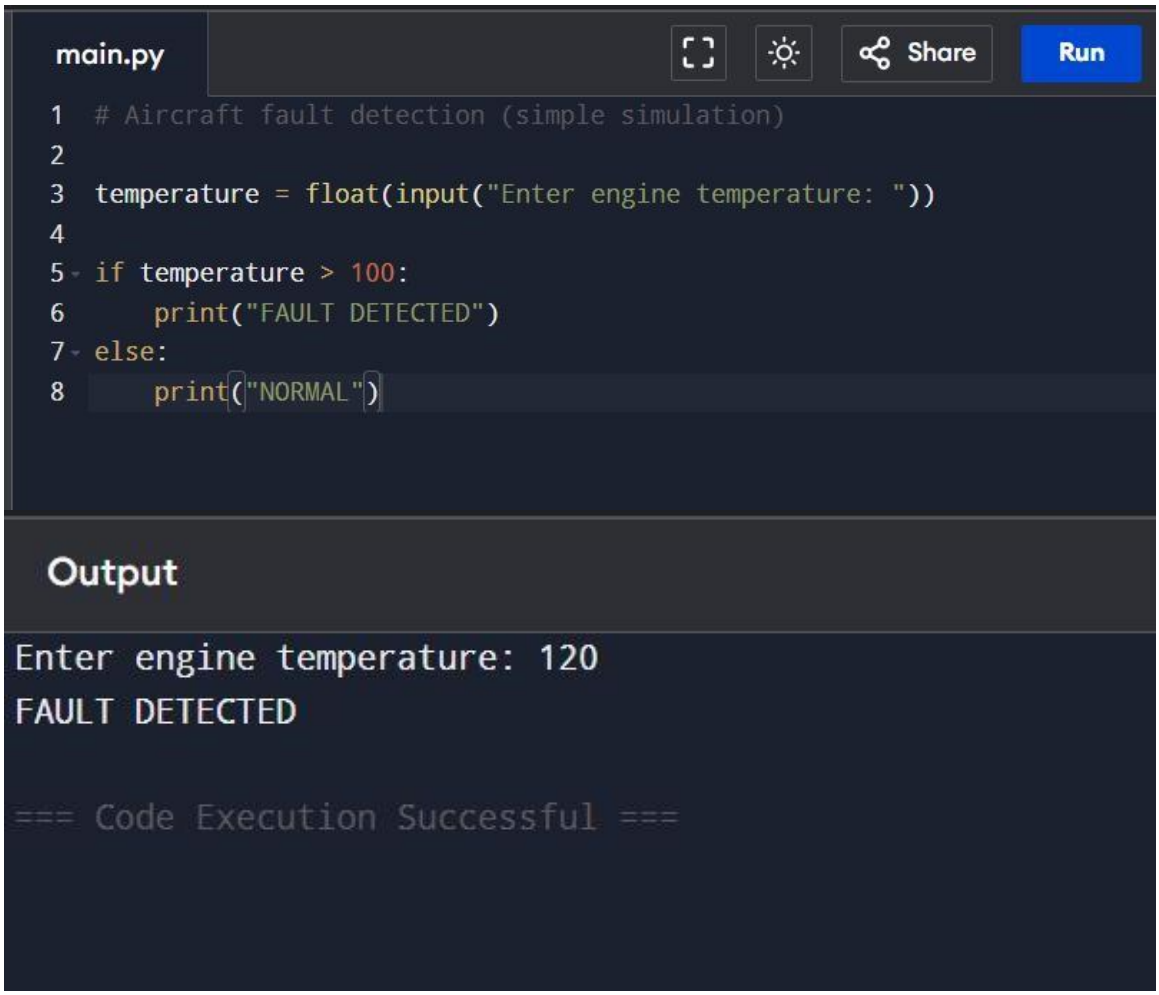
temperature = float(input("Enter engine temperature: "))

if temperature > 100:

print("FAULT DETECTED")

else:

print("NORMAL")



Task 6 – Ethical Use of AI-Generated Code

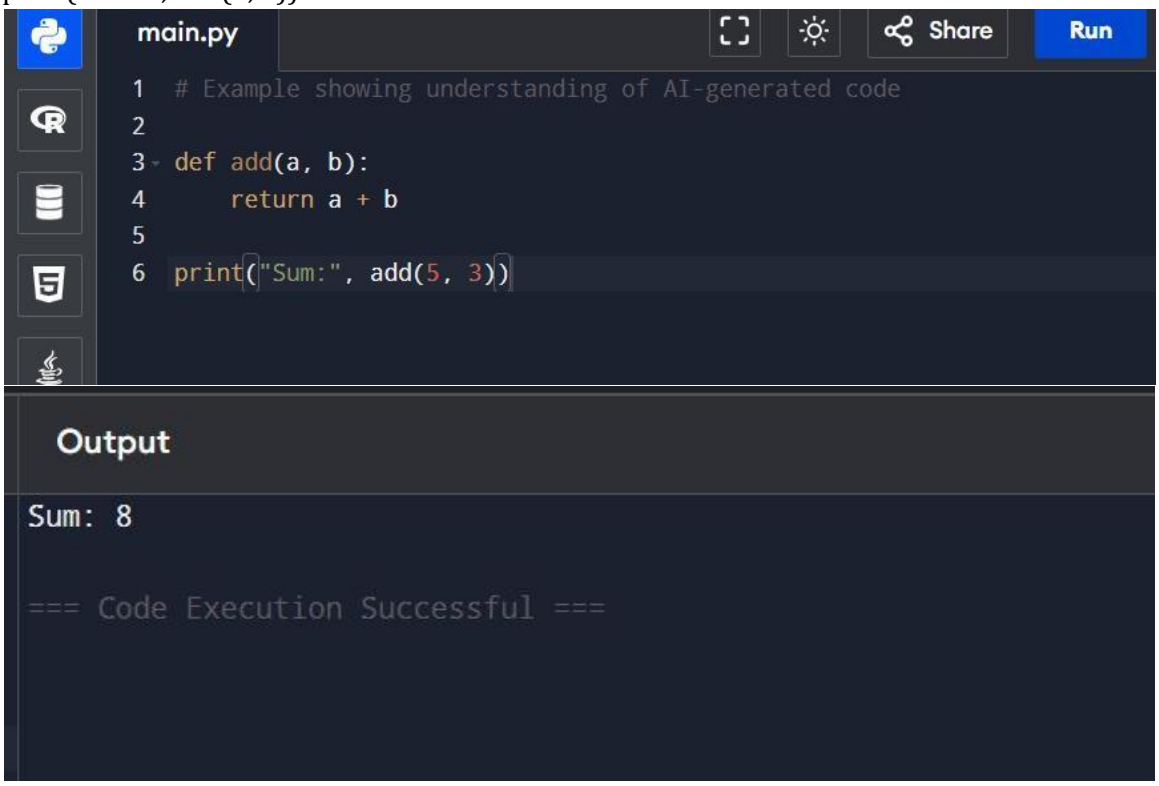
Explanation:

Using AI code without understanding is risky.

Best Practices:

Verify code, cite AI usage, avoid plagiarism.

Code:

Example showing understanding of AI-generated code

```
def add(a, b):
    return a + b


print("Sum:", add(5, 3))
```